
Python Learning Hub

Detailed architecture and implementation plan

A practical plan for a single-domain learning platform using FastAPI, Jinja2, HTMX, Flutter Web, PostgreSQL, and shared application services.

- Primary goal:** crawlable public learning content plus a rich authenticated app
- Backend boundary:** one FastAPI application
- Public pages:** server-rendered HTML with Jinja2 and selective HTMX
- Application pages:** Flutter Web under /app
- Data layer:** shared services and repositories over PostgreSQL

Contents

1	Executive summary	3
2	Product goals and architectural principles	3
2.1	What the product should feel like	3
2.2	Non-negotiable principles	3
2.3	Recommended technology choices	4
3	The two-experience model	4
3.1	Public website: Jinja2 plus HTMX	4
3.2	Application: Flutter Web	4
3.3	URL contract	5
4	End-to-end architecture	6
4.1	Logical architecture	6
4.2	What FastAPI manages	6
4.3	A request to a public topic	6
4.4	A request from Flutter	7
4.5	A transition from Flutter to public content	7
5	Repository structure	7
5.1	Recommended tree	7
5.2	Responsibilities by directory	8
6	FastAPI composition and routing	8
6.1	Application composition	8
6.2	Flutter fallback behavior	9
7	Shared services and data flow	9
7.1	One concept, two representations	9
7.2	Service responsibilities	9
7.3	Thin route handler example	10
7.4	Database direction	10
8	Templates, static assets, and accessibility	10
8.1	Template organization	10
8.2	Rendering rules	11
9	Authentication and authorization	11
9.1	Recommended first approach	11
10	Flutter Web boundary	11
10.1	Flutter source and generated output	11
10.2	Flutter route examples	12
10.3	API client discipline	12
11	Search engine optimization	12
11.1	Public-page checklist	12
12	Development and deployment workflow	13
12.1	Local development	13
12.2	Build script responsibilities	13

12.3 Production architecture	13
13 Testing strategy	14
13.1 Test layers	14
13.2 Release checks	14
14 Security and operational safeguards	14
15 Phased implementation roadmap	15
15.1 Phase 1: public learning site	15
15.2 Phase 2: accounts and lightweight learner actions	15
15.3 Phase 3: Flutter application	15
15.4 Phase 4: discussions and administration	15
15.5 Phase 5: real-time and scale features	15
16 Common mistakes to avoid	15
17 Definition of done	15
A Quick reference	16

1 Executive summary

The cleanest mental model is **one product with two experiences**: Jinja2 is the public, search-friendly website; Flutter is the authenticated application. FastAPI remains the single HTTP entry point, and both experiences use the same business services and repositories.

This plan turns that model into an implementable repository, URL contract, request flow, deployment process, and staged roadmap. It intentionally keeps the boundaries strict:

- Public, indexable content lives at ordinary URLs such as `/topics/python-functions`.
- Small authenticated enhancements use HTMX endpoints under `/web-components/...`
- Rich authenticated workflows live below `/app/...` and are rendered by Flutter Web.
- JSON endpoints live below `/api/v1/...`; WebSocket endpoints, if needed later, live below `/ws/...`
- Neither frontend talks directly to the database. Both call shared application services.

2 Product goals and architectural principles

2.1 What the product should feel like

The visitor should be able to discover and read Python learning content without an account. A signed-in learner should be able to continue into a more interactive application for progress, bookmarks, statistics, discussions, notifications, and other stateful features.

2.2 Non-negotiable principles

1. **One domain and one backend entry point.** Use a domain such as `pythonlearninghub.com`; let FastAPI decide which renderer handles each path.
2. **Presentation is separate from business logic.** Jinja2 and Flutter may render the same concept differently, but they must not implement separate copies of the rules.
3. **Public pages remain useful without JavaScript.** HTMX improves them; it must not be their only way to work.
4. **URLs reveal ownership.** The path prefix is a routing contract, not an informal convention.
5. **Build the smallest useful version first.** Redis, WebSockets, chat, and advanced moderation are later capabilities, not prerequisites for the public learning site.

2.3 Recommended technology choices

Area	Choice	Reason
HTTP backend	FastAPI	Clear routing, dependency injection, validation, async support, and API documentation.
Public rendering	Jinja2	Fast initial HTML, crawlability, and simple server-side composition.
Progressive interaction	HTMX	Small page updates without turning every public page into a single-page app.
Application UI	Flutter Web	Rich client-side state, navigation, dashboards, charts, and complex interaction.
Database	PostgreSQL	Strong relational model for users, content, progress, bookmarks, and moderation.
Persistence boundary	Repositories plus services	Keeps SQL and business decisions out of route handlers and widgets.

3 The two-experience model

3.1 Public website: Jinja2 plus HTMX

Use server-rendered pages for content that people may discover through search or share directly:

- home page, roadmap, categories, tags, tutorials, guides, and topic pages;
- public discussion pages and useful search results;
- explanatory content, examples, exercises, and related-topic navigation;
- simple actions such as bookmark, mark complete, reply, or filter, enhanced with HTMX when appropriate.

3.2 Application: Flutter Web

Use Flutter under `/app` for workflows whose value comes from client-side state and rich interaction:

- login, dashboard, profile, progress, bookmarks, and activity history;
- learner statistics, charts, filters, and multi-step workflows;
- richer discussions, notifications, chat, and administration when those features justify it.

Do not force every feature to exist in both frontends. A topic can have an SEO-friendly public page and an API representation for Flutter without requiring two separate user interfaces for every action.

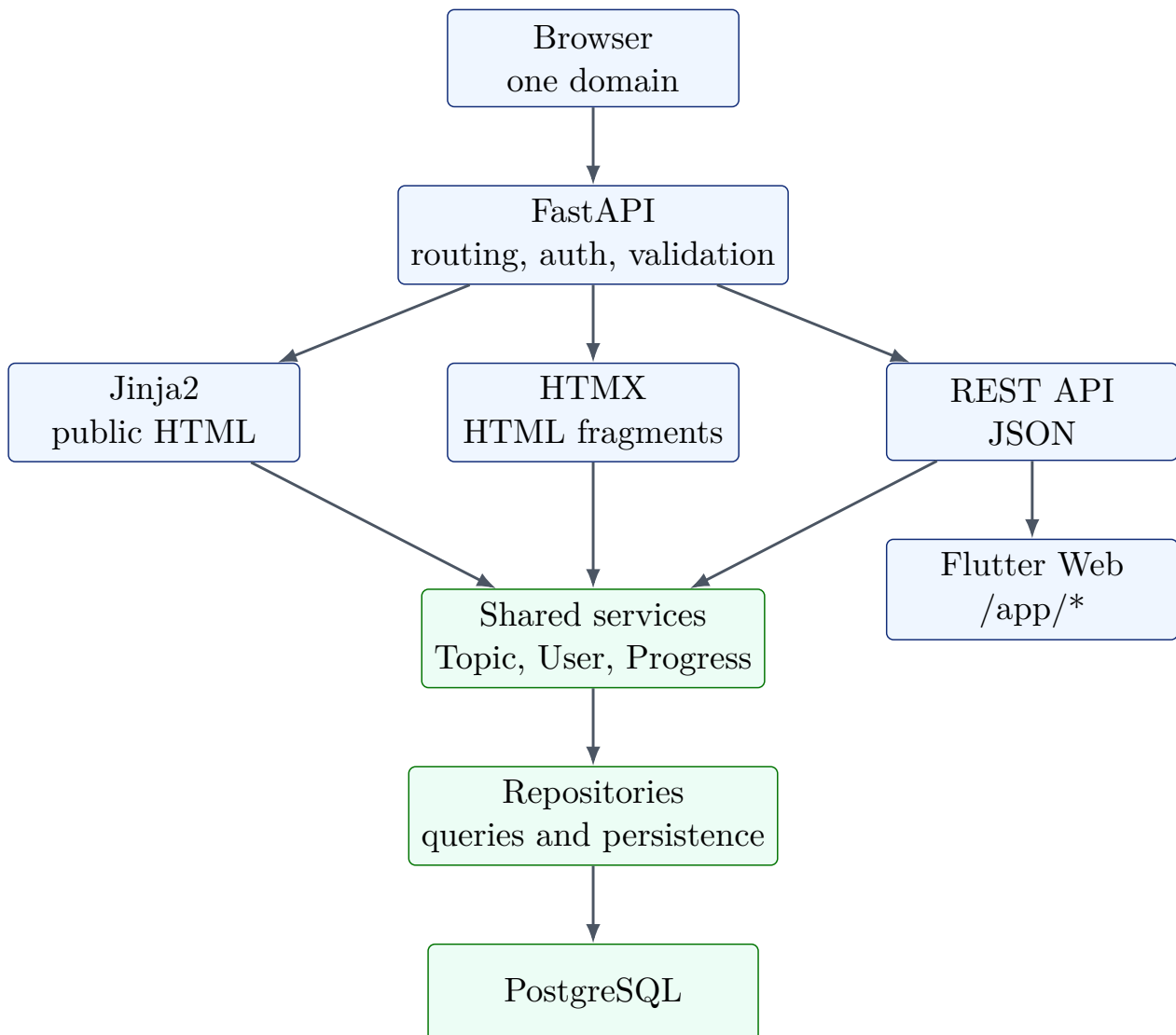
3.3 URL contract

Table 1: Stable URL ownership contract

Prefix or example	Owner	Purpose
/, /roadmap	Jinja2	Public landing and roadmap pages.
/topics/{slug}	Jinja2	Crawlable topic and tutorial content.
/categories/{slug}	Jinja2	Category landing pages.
/discussions/{id}	Jinja2	Public discussion pages.
/search	Jinja2	Search form and server-rendered results.
/web-components/...	HTMX/Jinja2	Small HTML fragments returned to public pages.
/app, /app/dashboard	Flutter Web	Authenticated client-side application routes.
/api/v1/...	FastAPI JSON	Data and mutation contract for Flutter and future clients.
/ws/...	FastAPI WebSocket	Optional real-time features introduced later.

4 End-to-end architecture

4.1 Logical architecture



4.2 What FastAPI manages

FastAPI is the composition point. It mounts routers, configures dependencies, serves templates and static files, exposes JSON endpoints, and returns the Flutter entry document for routes below `/app`. The route handlers should remain thin: authenticate, validate, call a service, and choose a response representation.

4.3 A request to a public topic

1. The browser requests `GET /topics/python-functions`.
2. FastAPI dispatches to the public topic router.
3. The router calls `TopicService.get_by_slug(...)`.
4. The service calls `TopicRepository`; the repository reads PostgreSQL.
5. The router passes the result to Jinja2 and returns complete HTML.

4.4 A request from Flutter

1. Flutter requests GET `/api/v1/progress` with the authenticated session.
2. FastAPI resolves the current user and validates the request.
3. The API router calls `ProgressService`.
4. The service uses the same repository layer and returns a response schema.
5. FastAPI serializes JSON; Flutter updates its state.

4.5 A transition from Flutter to public content

When a learner selects an SEO-oriented topic, Flutter can navigate the browser to `/topics/python-asyncio`. FastAPI then returns the public Jinja2 page. The learner has moved between two presentations without moving between domains or duplicating the content source.

5 Repository structure

5.1 Recommended tree

Listing 1: Repository layout

```
python-learning-hub/
|-- backend/
|   |-- app/
|   |   |-- main.py
|   |   |-- config.py
|   |   |-- dependencies.py
|   |   |-- routers/
|   |   |   |-- web/
|   |   |   |   |-- pages/
|   |   |   |   |   |-- home.py
|   |   |   |   |   |-- roadmap.py
|   |   |   |   |   |-- topics.py
|   |   |   |   |   |-- search.py
|   |   |   |   |-- components/
|   |   |   |   |   |-- progress.py
|   |   |   |   |   |-- bookmarks.py
|   |   |   |   |   |-- comments.py
|   |   |   |-- api/v1/
|   |   |   |   |-- auth.py
|   |   |   |   |-- topics.py
|   |   |   |   |-- progress.py
|   |   |   |   |-- bookmarks.py
|   |   |   |   |-- users.py
|   |   |   |-- websocket/chat.py
|   |   |-- services/
|   |   |-- repositories/
|   |   |-- schemas/
|   |   |-- models/
|   |   |-- templates/
|   |   |-- static/
|   |   |-- flutter/web/          # compiled Flutter output only
|   |-- migrations/
|   |-- pyproject.toml
|-- frontend/flutter_app/        # Dart source project
|-- scripts/build_flutter.py
|-- tests/
|-- README.md
-- .env.example
```

Keep Flutter source in `frontend/flutter_app`. Copy only its compiled `build/web` artifact into `backend/app/flutter/web`. This prevents Python tooling and Dart tooling from becoming one tangled project.

5.2 Responsibilities by directory

Table 2: Directory responsibilities

Directory	Responsibility
<code>routers/</code>	HTTP and WebSocket entry points; no raw SQL.
<code>services/</code>	Business rules, orchestration, authorization decisions, and transaction boundaries.
<code>repositories/</code>	Database queries and persistence operations.
<code>schemas/</code>	Request and response contracts; keep API schemas separate from database records when useful.
<code>templates/</code>	Full pages and reusable HTML fragments.
<code>static/</code>	CSS, browser JavaScript, icons, and public images.
<code>flutter/web/</code>	Generated Flutter Web deployment files; do not edit by hand.
<code>migrations/</code>	Versioned PostgreSQL schema changes.
<code>scripts/</code>	Repeatable build, seed, export, and deployment helpers.
<code>tests/</code>	Unit, integration, route, API, and browser-facing tests.

6 FastAPI composition and routing

6.1 Application composition

The central module should make the routing boundary obvious. The following is intentionally schematic; exact framework versions and database libraries can be selected during implementation.

Listing 2: Composition point in `main.py`

```
from fastapi import FastAPI

from app.routers.api.v1 import router as api_router
from app.routers.web.pages import router as web_router
from app.routers.web.components import router as component_router
from app.flutter import mount_flutter_web

app = FastAPI(title="Python Learning Hub")

app.include_router(web_router)
app.include_router(component_router, prefix="/web-components")
app.include_router(api_router, prefix="/api/v1")
mount_flutter_web(app, prefix="/app")
```

6.2 Flutter fallback behavior

Flutter uses client-side navigation. A request to `/app/dashboard` therefore needs the same Flutter entry document as `/app/`. The mounting helper should:

1. serve real generated assets such as JavaScript, icons, and canvaskit files;
2. return Flutter's `index.html` for an application route that is not a physical asset;
3. never catch `/api`, public pages, or HTMX endpoints;
4. use an explicit prefix so a missing asset cannot silently become a public HTML page.

Listing 3: Conceptual Flutter route fallback

```
@router.get("/{path:path}")
async def flutter_route(path: str):
    requested = flutter_root / path
    if requested.is_file():
        return FileResponse(requested)
    return FileResponse(flutter_root / "index.html")
```

The fallback is deliberately scoped below `/app`. A global catch-all route is a common source of broken public pages, incorrect 404 behavior, and difficult debugging.

7 Shared services and data flow

7.1 One concept, two representations

For a topic with slug `python-functions`, expose two representations:

Request	Response	Shared path
GET/topics/ python-functions	Complete HTML	Topic router → TopicService → TopicRepository
GET/api/v1/topics/ python-functions	JSON	API router → TopicService → TopicRepository
POST /web-components/...	HTML fragment	Component router → ProgressService → repository

7.2 Service responsibilities

Services should answer product questions rather than framework questions. Examples include:

- **TopicService**: locate, publish, search, and relate learning content;
- **AuthService**: register, authenticate, rotate sessions, and resolve account state;
- **ProgressService**: mark a lesson complete, calculate progress, and record activity;
- **BookmarkService**: add, remove, and list saved items;
- **DiscussionService**: create, edit, moderate, and retrieve discussion content.

7.3 Thin route handler example

Listing 4: A public route that delegates to a service

```
@router.get("/topics/{slug}", response_class=HTMLResponse)
async def topic_page(request: Request, slug: str, service: TopicService = Depends()):
    topic = await service.get_published_by_slug(slug)
    if topic is None:
        raise HTTPException(status_code=404)
    return templates.TemplateResponse(
        request=request,
        name="topics/detail.html",
        context={"topic": topic},
    )
```

7.4 Database direction

Start with a relational schema that reflects the learning domain. A practical first set is:

Table 3: Initial domain tables

Table	Purpose and key relationships
users	Identity, account status, role, and timestamps.
topics	Slug, title, body, summary, category, difficulty, publication state, and SEO metadata.
roadmap_nodes	Ordered learning path and prerequisite relationships.
progress	User-to-topic completion state, percentage, and last activity.
bookmarks	User-to-topic saved items with uniqueness constraint.
discussions	Public questions or conversations associated with a topic.
discussion_replies	Replies, authorship, moderation state, and timestamps.
activity_events	Optional append-only learning events for statistics.

8 Templates, static assets, and accessibility

8.1 Template organization

Listing 5: Template tree

```
templates/
|-- base.html
|-- home.html
|-- roadmap.html
|-- topics/
|   |-- detail.html
|   '-- list.html
|-- discussions/detail.html
|-- search/results.html
'-- components/
    |-- progress_button.html
    |-- bookmark_button.html
    |-- search_results.html
    '-- flash_message.html
```

8.2 Rendering rules

- Keep page-level templates responsible for layout and metadata.
- Keep component templates small enough to return as an HTMX response.
- Use semantic headings in order; do not use font size as a substitute for document structure.
- Provide visible focus states, keyboard access, labels for form controls, and useful empty/error states.
- Treat user-authored discussion content as untrusted; escape by default and sanitize only through a deliberate policy.

9 Authentication and authorization

9.1 Recommended first approach

Use a secure, same-site session cookie for the browser-facing product. FastAPI dependencies can resolve the current user for both HTML routes and API routes. If a separate mobile client is required later, add a token-based API flow without changing the service boundary.

1. Hash passwords with a modern password-hashing library; never store plaintext passwords.
2. Set cookies as Secure in production, HttpOnly, and an intentional SameSite policy.
3. Add CSRF protection to cookie-authenticated state-changing browser requests, including HTMX requests.
4. Check authorization in services or dedicated policy functions, not only in templates.
5. Make admin and moderation roles explicit and auditable.

Listing 6: Conceptual dependency usage

```
@router.post("/progress/{topic_id}")
async def mark_complete(
    topic_id: int,
    user: CurrentUser,
    service: ProgressService = Depends(),
):
    result = await service.mark_complete(user.id, topic_id)
    return templates.TemplateResponse(
        request=request,
        name="components/progress_button.html",
        context={"progress": result},
    )
```

10 Flutter Web boundary

10.1 Flutter source and generated output

The Dart project remains a normal Flutter project. It owns widgets, navigation, state management, API clients, and application styling. The backend owns the compiled artifact only.

Listing 7: Development and production relationship

```

frontend/flutter_app/          # source, edited by developers
  |
  | flutter build web
  v
frontend/flutter_app/build/web/ # generated deployment artifact
  |
  | copy during build
  v
backend/app/flutter/web/        # served below /app

```

10.2 Flutter route examples

Route	Screen responsibility
/app/login	Sign-in and account recovery.
/app/dashboard	Continue learning, summary progress, and recent activity.
/app/profile	Account settings and preferences.
/app/progress	Detailed roadmap and topic completion state.
/app/bookmarks	Saved topics and guides.
/app/discussions	Rich discussion participation if the feature needs it.
/app/admin	Moderation and content administration, protected by role.

10.3 API client discipline

Flutter should use a small typed API client. Keep endpoint paths, serialization, error mapping, and authentication behavior in one place. Widgets should not assemble raw URLs or interpret database-shaped responses directly.

11 Search engine optimization

11.1 Public-page checklist

Every indexable page should have:

- a unique, descriptive title and meta description;
- one clear H1 and a logical heading hierarchy;
- canonical URL handling;
- meaningful internal links between related topics;
- server-rendered content in the initial response;
- useful 404 and no-result pages;
- structured data only when it accurately describes the page;
- a sitemap and robots policy that reflect publication state.

The SEO rule is simple: if a page is valuable to someone who has not signed in, make it a real Jinja2 URL. Flutter can link to it, but search engines should not need to boot Flutter to read it.

12 Development and deployment workflow

12.1 Local development

Run the backend and Flutter source project independently during development. The public website and API can run on FastAPI while Flutter uses its normal development server. Configure the API base URL so the Flutter client can call the local backend.

Listing 8: Conceptual local workflow

```
Terminal A: backend development server on localhost:8000
Terminal B: Flutter development server
Database: local PostgreSQL instance or development container
```

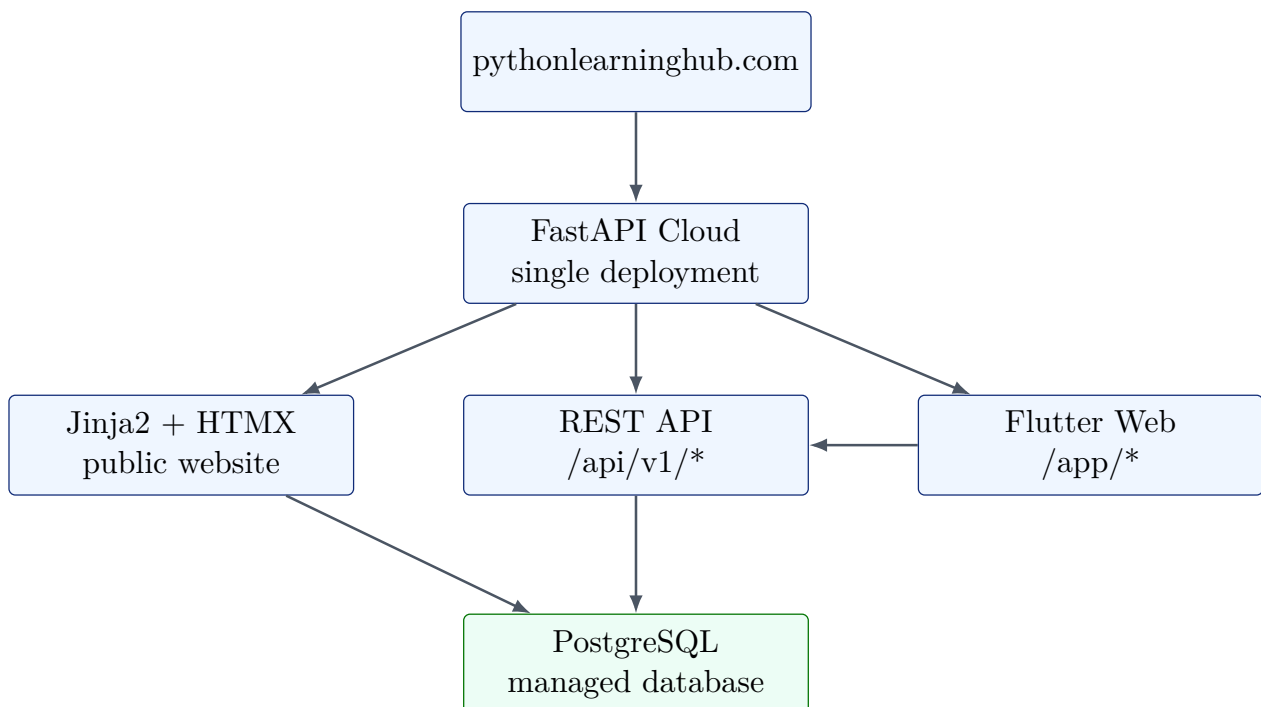
12.2 Build script responsibilities

Create `scripts/build_flutter.py` or an equivalent task runner that:

1. runs the approved Flutter Web build command;
2. validates that `build/web/index.html` exists;
3. replaces the generated backend Flutter directory in a controlled way;
4. copies the build artifact into `backend/app/flutter/web`;
5. reports the build hash and output path;
6. fails before deployment if required files are missing.

Generated output should be reproducible and disposable. Do not hand-edit generated JavaScript or put secrets in Flutter assets; Web assets are publicly downloadable.

12.3 Production architecture



13 Testing strategy

13.1 Test layers

Table 4: Proportional testing plan

Layer	Examples	Purpose
Unit	slug rules, progress calculation, authorization policies	Fast feedback on business rules.
Repository	topic lookup, unique bookmark, pagination	Verify SQL and database assumptions.
Service	publish topic, mark complete, moderate reply	Verify orchestration and transaction behavior.
Route	status codes, template context, API schema	Verify HTTP contracts.
Browser	public page renders without JS; HTMX swaps target	Protect the public experience.
Flutter	navigation, API error states, auth refresh	Protect application workflows.
End-to-end	login to progress to public topic link	Verify the boundaries work together.

13.2 Release checks

Before production deployment, confirm:

- all migrations apply cleanly to a fresh database;
- public pages return useful HTML with JavaScript disabled;
- direct refreshes of `/app/dashboard` and other Flutter routes work;
- unknown `/app` assets return an appropriate response;
- API errors are consistent and do not leak database details;
- cookies, CSRF protection, CORS, and trusted-host settings match the deployment;
- the generated Flutter artifact is present in the deployment package;
- no secrets, local paths, or debug settings are included in the release.

14 Security and operational safeguards

- Validate all input at the API boundary and enforce domain rules in services.
- Parameterize database queries; never interpolate user input into SQL.
- Apply rate limits to authentication, search, comments, and expensive endpoints.
- Limit upload size and file type if user uploads are introduced.
- Log request identifiers, authentication failures, moderation actions, and unexpected errors without logging secrets.
- Back up PostgreSQL and test restoration before relying on it.
- Add health checks that verify application availability and database connectivity.
- Keep dependency updates regular and review security advisories.

15 Phased implementation roadmap

15.1 Phase 1: public learning site

Build the smallest useful product first:

- FastAPI, PostgreSQL, migrations, configuration, and deployment skeleton;
- Jinja2 base layout, home page, roadmap, topics, categories, and search;
- service and repository boundaries for content;
- metadata, sitemap, 404 page, accessibility, and responsive styling;
- tests for public routes and content queries.

15.2 Phase 2: accounts and lightweight learner actions

- registration, login, logout, password recovery, and session handling;
- progress and bookmarks through services and repositories;
- HTMX buttons for simple actions on public pages;
- API schemas and versioned routes for the same data.

15.3 Phase 3: Flutter application

- create the Flutter source project and typed API client;
- implement login, dashboard, profile, progress, and bookmarks;
- add the reproducible build-and-copy workflow;
- mount the compiled output under `/app` and test direct route refreshes.

15.4 Phase 4: discussions and administration

Add public discussions, replies, reports, moderation state, admin screens, audit events, and role-based access. Decide feature by feature whether simple actions remain HTMX or move into Flutter.

15.5 Phase 5: real-time and scale features

Only after the core product is stable should you add Redis, WebSockets, chat, notifications, background jobs, search indexing, caching, or more advanced analytics.

16 Common mistakes to avoid

1. Putting Dart source files inside the Python application directory.
2. Creating different business rules for Jinja2 and Flutter.
3. Making public topic content depend on Flutter booting successfully.
4. Adding a global catch-all route that steals 404s from the public website.
5. Returning database models directly as API contracts.
6. Putting raw SQL in route handlers or Flutter widgets.
7. Introducing WebSockets and Redis before the basic learning workflow is validated.
8. Treating generated Flutter output as hand-maintained source.
9. Forgetting that every browser-delivered Flutter asset is public.

17 Definition of done

The first production-ready milestone is complete when:

- a visitor can browse and search useful Python content with JavaScript disabled;
- the same domain serves public pages, HTMX fragments, JSON, and Flutter below clear prefixes;
- a signed-in learner can record progress and bookmarks;
- Jinja2 and Flutter consume the same service and repository rules;

- direct navigation and refresh work for public and Flutter URLs;
- automated tests cover the primary content and authentication flows;
- deployment can rebuild the Flutter artifact instead of relying on manual file edits;
- logging, backups, migrations, security settings, and error handling are documented.

The core rule to keep visible while implementing is: **one domain, one FastAPI backend, two deliberate presentation layers, shared services, and a staged roadmap**. That gives the project the SEO benefits of server-rendered content without giving up the richer interactions that Flutter is good at.

A Quick reference

Question	Default decision
Where does public content render?	FastAPI plus Jinja2.
Where do small page updates render?	Jinja2 fragments returned through HTMX.
Where does the rich authenticated UI render?	Flutter Web below <code>/app</code> .
Where does Flutter get data?	FastAPI JSON endpoints below <code>/api/v1</code> .
Where does business logic live?	Shared services, called by both route families.
Where does SQL live?	Repositories only.
Where does Flutter source live?	<code>frontend/flutter_app</code> .
What is deployed for Flutter?	Generated <code>build/web</code> copied to the backend artifact.
What should be built first?	Public content, search, authentication, progress, and bookmarks.
What comes later?	Rich discussions, admin, Redis, WebSockets, chat, and advanced analytics.